

Project 1: Containerized News Search Application

Parijat Das · Connor Fleischman

Marist University, Poughkeepsie, NY, USA

MSCS 621 Cloud Computing · Fall 2026

Project 1 · 4 October, 2026

Abstract: This project presents a Go-based command-line application for retrieving news articles through NewsAPI, caching search results, and processing concurrent requests. Users specify a search topic, a lookback period, and a maximum article count. The application evaluates cached query coverage and freshness before issuing an external request, and it persists article and coverage records in SQLite. Goroutines and channels support concurrent request processing, while a per-query lock limits redundant retrievals for overlapping cache misses within a process. A multi-stage Docker build packages the application in a minimal runtime image. In a local benchmark with a fixed 50 ms mock-service delay, the median latency was 0.926 ms for a single warm-cache search and 62.647 ms for a single cache miss. Distinct cold searches achieved 118.190 searches per second at a concurrency level of 20. The measured runtime image occupied 5,277,856 bytes in the local Docker image store. These results demonstrate the benefit of cache reuse under controlled conditions and characterize the application as a compact, single-host deployment. They do not establish performance against the live NewsAPI service or across multiple hosts.

Index Terms: Caching, containerization, Go, news retrieval, SQLite.

I. INTRODUCTION

Repeated news searches can incur unnecessary network requests when users request overlapping topics and publication periods. A persistent cache can reuse previously retrieved articles, but reuse must account for both data freshness and query coverage. A cached response may be unsuitable if the requested date range expands or the requested article count exceeds the available results.

This project implements a command-line interface (CLI) that combines NewsAPI retrieval with persistent caching, concurrent request processing, and Docker-based execution. NewsAPI provides an HTTP application programming interface (API) for searching and retrieving news articles [1]. The application accepts individual searches or a batch of requests and identifies whether each result originated from the cache or the external service.

The project evaluates how cache reuse and concurrent processing affect latency, throughput, and external request counts in a controlled local environment. Its contribution is the integration and evaluation of established technologies for a cloud-computing application. The implementation also examines a deployment tradeoff: replacing a separate database service with embedded SQLite to simplify the runtime and reduce its storage footprint. Unlike the large-scale service setting discussed in Dean and Barroso's work, this project focuses on a single-host CLI. The application-specific contribution is the integration of coverage-aware news caching, batch request processing, and a measured embedded SQLite deployment.

II. SYSTEM DESIGN

A. Application Structure

The implementation separates command processing, data representation, retrieval, cache policy, and persistence. The entry point, `main.go`, validates CLI arguments and invokes the request pipeline. The file

models/news.go defines article and response structures. The NewsAPI client and request normalization reside in newsapi/newsapi.go, while newsapi/cache.go implements coverage checks, request fan-out, deduplication, and result merging. The persistence layer in newsapi/sqlite.go initializes the database schema and writes article and coverage records within transactions.

The CLI accepts a topic through `-topic` or its legacy alias, `-q`. The `-days` argument specifies the lookback window (`-days 2` means today and yesterday using UTC calendar dates), and `-articles` sets the maximum number of results. Topic searches use the `/v2/everything` endpoint, and the code removes the country parameter for that endpoint, so `-country` does not filter topic searches, matching the endpoint's documented parameters. Batch execution reads requests from a JSON file supplied through `-batch`. Output includes article titles, sources, authors, publication dates, descriptions, URLs, and the retrieval source. When the available result count is below the requested limit, the application reports the difference.

B. Cache Policy and Persistence

Cache lookup accounts for the normalized topic, country, and requested date range. A 15-minute lifetime limits the reuse of stale results. The application checks stored coverage and available article counts before returning cached data. If the requested time window or result count is insufficiently covered, the application retrieves additional data and merges the results. It should be noted that increasing the article limit does not always trigger another API call because a prior fetch may already have cached enough articles since retrieval uses pages of 100. Article URLs serve as deduplication identifiers, with a content-derived fallback when a URL is unavailable.

The articles table stores topic and country information, article identity, publication time, and complete article JSON. The coverage table records the covered date range, fetch time, requested limit, and an exhausted flag. Separating articles from coverage metadata allows the application to track the scope of a previous retrieval alongside its stored results. Persistent storage is enabled through `DATABASE_PATH`; the container deployment uses `/data/news.db` on a named volume mounted at `/data`.

C. Concurrent Request Processing

`ProcessRequestsWithSource` starts a goroutine for each request and sends `RequestResult` values and errors through separate channels. Goroutines and channels are defined by the Go language specification [2], while synchronization behavior is described by the Go memory model [3]. Independent searches can overlap rather than waiting for all preceding requests to finish. A per-query lock coordinates overlapping cache misses within the process so that a subsequent request can reuse data obtained by an earlier request.

SQLite is configured to use write-ahead logging (WAL), a bounded busy timeout, synchronous writes, and short transactions. WAL permits readers and a writer to operate concurrently, although SQLite still allows only one writer at a time [4], [5]. The application does not hold a write transaction open during an HTTP request. This design limits the time spent holding database write locks. The in-process query lock and SQLite transaction handling serve different purposes: the former limits redundant retrievals, while the latter coordinates access to persistent data. Cross-process request coalescing is not established by the in-process lock.

D. Container Packaging and Database Selection

The Docker build compiles the executable in a builder stage, compresses it with UPX, and copies the runtime binary and certificate authority bundle into a final scratch image. Multi-stage builds allow build tools and intermediate artifacts to remain outside the final runtime image [6]. Docker Compose provides the execution configuration and persistent database volume.

The final implementation replaces the earlier MongoDB-oriented architecture with embedded SQLite. This removes the need to deploy a separate MongoDB service image and supports a compact single-host configuration. The project records this migration in README.md and docs/docker_image_optimization.md. The available measurements establish the size of the SQLite-based artifact.

III. EVALUATION METHOD

A. Functional and Concurrency Tests

The test suite exercises cache misses, cache hits, invalid batch input, concurrent requests, output formatting, and SQLite persistence. A cache-miss fixture checks that an upstream request occurs and that its results are stored. A cache-hit fixture checks result reuse. Invalid-batch tests check that validation fails before retrieval. TestConcurrentRequestsFixture verifies overlapping request execution and expected results using a mock fetch function; cache concurrency tests verify shared-cache behavior. Golden-output tests compare CLI output with expected text. SQLite tests exercise repeated access, overlapping operations, and failed writes.

The project identifies metrics/test_results.jsonl and metrics/race_results.jsonl as the stored outputs of the test and race-detector runs. The recorded results show 57 named tests passed in the normal run and 57 in the race-detector run, with zero failures or skipped named tests. This finding applies to the executed tests: Go’s race detector observes runtime accesses and cannot establish the absence of races in unexecuted paths [7]. Build and test commands are provided in Appendix A.

B. Benchmark Configuration

The benchmark harness in metrics/harness/main.go invokes the application fetch path and its goroutine-and-channel request processing. The script metrics/run_metrics.sh drives execution and captures measurements. A local mock HTTP server introduces a fixed 50 ms delay, allowing cache and concurrency behavior to be measured without variation from the live NewsAPI service. Temporary SQLite storage supports the warm-cache and cold-search scenarios.

The measurement procedure specifies one warm-up run followed by five measured repetitions per scenario. The reported environment comprised Go 1.25.14, GCC 15.2.0, Docker Engine 29.8.0, and Docker Desktop 4.91.0, with Linux/amd64 execution on a Windows host. The benchmark records latency percentiles, maximum latency, throughput, error counts, and API-call counts. Table I reproduces the available scenario summaries; p50 denotes median latency.

IV. RESULTS AND DISCUSSION

A. Cache and Concurrency Performance

TABLE I
REPORTED LOCAL BENCHMARK RESULTS

Scenario	Concurrency	Completed	API calls	Searches/s	p50 (ms)
Warm cache	1	100	0	1044.035	0.926
Warm cache	20	100	0	1044.043	12.259
API fetch / cache miss	1	100	100	15.451	62.647

Scenario	Concurrency	Completed	API calls	Searches/s	p50 (ms)
Distinct cold searches	20	100	100	118.190	94.894
Identical cold searches	20	100	5	224.394	67.167

All five displayed scenarios recorded zero errors. At concurrency one, the warm-cache median latency was 0.926 ms, compared with 62.647 ms for a cache miss. The ratio of these medians is approximately 67.7. This comparison reflects the avoided mock-service delay and associated retrieval and persistence work; it is not an estimate of live-service acceleration.

Distinct cold searches at concurrency 20 achieved 118.190 searches/s, compared with 15.451 searches/s for the single-request cache-miss scenario. The measurements are consistent with overlapping retrieval work. For identical cold searches, the table records five upstream calls for 100 completed searches, demonstrating substantial reuse within the measured workload. The harness uses new topic names for each cold wave. At concurrency 20, identical searches produce one API call per wave across five measured repetitions, explaining the five API calls.

Increasing warm-cache concurrency from one to 20 left throughput approximately unchanged at 1044 searches/s, while median latency increased from 0.926 to 12.259 ms. Thus, this workload did not exhibit proportional throughput scaling with concurrency. The measurements do not isolate whether database access, locking, scheduling, or another component imposed the limit.

The full benchmark summary reports 1,600 measured operations without errors (16 scenario and concurrency combinations with 100 measured searches each). Table I acts as a selected summary showing 500 completed searches. Although the harness records p95 and p99 latency, those values are not included here. Tail latency warrants separate analysis because median latency alone can obscure slow requests, a concern discussed by Dean and Barroso [8].

B. Runtime Artifact Size

TABLE II
MEASURED SQLITE DEPLOYMENT ARTIFACTS

Artifact		Size (bytes)
Executable after UPX compression		2,470,712
Runtime image in local Docker image store		5,277,856
Docker image export archive (metrics artifact)		2,595,328
Submitted Docker archive (project1-go-app.tar)	2,588,672	

The runtime image occupied approximately 5.28 MB using decimal units. Executable size, local image-store size, and export-archive size measure different artifacts and should not be compared as interchangeable values. These measurements characterize deployment storage rather than runtime memory use. The available measurements in runtime.json show Docker cache-hit invocation times with a median of 629.882 ms (including container creation/removal, startup, lookup, and output). Packed

executable process times were 0.28–0.29 seconds (excluding Docker startup), with a peak RSS of 7,888–8,016 KiB. These are complete invocation measurements rather than isolated startup times. The submitted archive `project1-go-app.tar` contains `project1-go-app:latest` and is 2,588,672 bytes. The 2,595,328-byte export measurement belongs to the earlier `project1-metrics:latest` artifact and is not the submitted archive's size. A controlled comparison with the former Go-and-MongoDB deployment could be useful to quantify the effect of the migration on total image storage, startup latency, or memory consumption.

V. LIMITATIONS AND FUTURE WORK

The evaluation uses a deterministic mock service on one host. It excludes live-service latency variation, rate limiting, external failures, and changes in news availability. The results therefore characterize the tested application paths under controlled conditions. They do not establish production performance or multi-host scalability.

SQLite simplifies deployment but constrains the architecture to local persistent storage in the configuration evaluated. WAL relies on shared-memory coordination and does not support clients on different machines sharing a database over a network filesystem [4]. Concurrent writes remain serialized. In addition, per-query locking coordinates retrievals only within an application process; multiple container invocations may still issue redundant upstream requests.

Further evaluation should reconcile the benchmark sample counts, document cache reset behavior between repetitions, and report p95 and p99 latency alongside the median. CPU and memory measurements, startup timing, and a comparable MongoDB baseline would support a broader deployment comparison. Separate-process database testing already exists (`TestSQLiteProcesses` checks six concurrent writer processes, duplicate prevention, interrupted-transaction recovery, and integrity). Container-level concurrent load testing remains a valid future task.

VI. CONCLUSION

The project integrates Go concurrency, NewsAPI retrieval, SQLite persistence, and Docker packaging into a cached news-search CLI. Coverage and freshness checks allow stored results to satisfy repeated searches, while concurrent processing overlaps independent retrievals. Under a fixed-delay local mock service, a warm-cache search had a median latency of 0.926 ms, and distinct cold searches achieved 118.190 searches/s at concurrency 20. The SQLite-based runtime image occupied 5,277,856 bytes in the local Docker image store. The evaluation supports the use of this design for a compact single-host deployment, with further measurements needed to establish live-service behavior, tail latency, and performance across processes.

AI USE STATEMENT

The project maintains development prompt logs in `prompts/connor_prompts.md` and `prompts/das_prompts.md`. Connor's five logged prompts have no associated AI names or versions. Furthermore, the development record mentions the use of Gemini 3.1 Pro, GPT-6 Astra/Luna, and GPT-6.1 Sol. An AI model (with exact prompts and model version unrecorded) was also used on 4 October, 2026, to convert a bullet-style report into this formal technical prose, organize the report using IEEE-style sections and numbered references, check public technical documentation, qualify unsupported claims, and format the document for refinement and validation in Google Docs.

REFERENCES

- [1] NewsAPI, “Documentation.” Accessed: Sept. 24, 2026. [Online]. Available: <https://newsapi.org/docs>
- [2] The Go Authors, “The Go Programming Language Specification.” Accessed: Sept. 14, 2026. [Online]. Available: <https://go.dev/ref/spec>
- [3] The Go Authors, “The Go Memory Model.” Accessed: Sept. 28, 2026. [Online]. Available: <https://go.dev/ref/mem>
- [4] SQLite, “Write-Ahead Logging.” Accessed: Oct. 1, 2026. [Online]. Available: <https://sqlite.org/wal.html>
- [5] SQLite, “Transaction.” Accessed: Oct. 1, 2026. [Online]. Available: https://sqlite.org/lang_transaction.html
- [6] Docker, “Multi-stage builds.” Accessed: Sept. 29, 2026. [Online]. Available: <https://docs.docker.com/build/building/multi-stage/>
- [7] The Go Authors, “Data Race Detector.” Accessed: Sept. 28, 2026. [Online]. Available: https://go.dev/doc/articles/race_detector
- [8] J. Dean and L. A. Barroso, “The tail at scale,” *Commun. ACM*, vol. 56, no. 2, pp. 74–80, Feb. 2013, doi: 10.1145/2408776.2408794.

APPENDIX A BUILD AND EXECUTION

Native builds require Go 1.25.0 or later and a CGO-compatible C compiler. Docker provides the reference build workflow. Configure NewsAPI credentials using the repository instructions before performing live searches. Persistent database operation requires DATABASE_PATH; the container configuration uses /data/[news.db](#). If using the github repository and not the direct project itself, navigate into the Project1 directory and configure the environment:

```
cd Project1
```

Then copy the .env.example to create the template for your own .env:

```
Copy-Item .env.example .env
```

Enter your NewsAPI key in .env. Run the following commands:

A. Build and Run

```
docker compose -p project1 build go-app
```

```
docker compose -p project1 run --rm go-app -topic "cloud computing" -days 7 -articles 5
```

For batch execution, mount the example JSON input and pass its container path. Label the multiline command as Bash, or use a single line for PowerShell since its `` continuation syntax does not work in PowerShell:

```
docker compose -p project1 run --rm \
  -v "./searches.example.json:/app/searches.example.json:ro" \
  go-app -batch /app/searches.example.json
```

B. Test Execution

```
docker build -f dockerfile --target build -t project1-go-build .
```

```
docker run --rm -e DATABASE_PATH= project1-go-build go test ./... -count=1
```

```
docker run --rm -e DATABASE_PATH= project1-go-build go test -race ./... -count=1
```

Individual fixtures can also be executed in a configured Go build environment:

```
go test ./newsapi -run '^TestCacheMissFixture$' -v -count=1
```

```
go test ./newsapi -run '^TestCacheHitFixture$' -v -count=1
```

```
go test . -run '^TestInvalidBatchFixture$' -v -count=1
```

```
go test ./newsapi -run '^TestConcurrentRequestsFixture$' -v -count=1
```

```
go test . -run '^TestPrintArticlesGoldenOutput$' -v -count=1
```

Example inputs are located in searches.example.json, newsapi/testdata/cache-case.json, newsapi/testdata/concurrent-case.json, and testdata/invalid-batch-case.json. Expected output files include testdata/expected/cache-hit.txt and testdata/expected/cache-miss.txt. Benchmark execution is managed by metrics/run_metrics.sh and metrics/harness/main.go.